

METRIC_GOVERNANCE_GUIDE_PLAIN

Helios Metric Governance — In Plain English

Plain-language companion to `docs/architecture/METRIC_GOVERNANCE_GUIDE.md`. For the exact spec, read that / its PDF.

In one sentence

Every metric in Helios is defined exactly once, in one YAML file — so the AI can only ever name a metric, never invent one or hand-write the SQL behind it.

Why this matters to you

Helios makes causal-sounding claims about *why* a funnel moved. Those claims are only believable if every number traces to one agreed definition. The semantic layer (`semantic_layer.yaml`) is that single source of truth. The payoff is concrete: **a metric that isn't in the registry cannot be queried, charted, or named in a brief** — and a hallucinated column literally has no template to appear in. This is how you get to "0 hallucinated columns, 100% governed SQL" as a *structural* guarantee, not a hope. If you're adding or changing a metric, this is the rulebook.

The big ideas, simply

The three invariants:

- **I1 — No metric exists outside the registry.** Not in prompts, not in Python, not in a dashboard. Two definition sites = two truths = silent drift.
- **I2 — `build_query` is the only path to SQL.** Generation (`semantic-mcp`) is separated from execution (`warehouse-mcp`), so every query is governed, cost-checked, and reconciled before a byte is scanned.
- **I3 — Zero hallucinated columns.** `build_query` only interpolates registry-defined templates, so a made-up column can't appear.

The chokepoint chain: agent picks canonical names → `semantic-mcp.build_query` → `warehouse-mcp.dry_run` (≤ 5 GiB, **G3**) → `warehouse-mcp.run_query` → `warehouse-mcp.reconcile` ($\leq 0.5\%$ drift, **G4**) → `stats-mcp` for all math (**G2**) → Critic refutes using guardrails. Bypass it anywhere and the finding is void.

The Simpson's-paradox defense, baked in: rates are always `SUM(numerator) / SUM(denominator)` *after* grouping — never an average of per-segment ratios. This is non-negotiable and shows up everywhere.

What you actually build / how it works

Every metric carries a full field schema — no partial entries. Key fields:

- `metric_name` (snake_case, globally unique) — the *only* token agents pass.
- `type` — one of `count` | `sum` | `ratio` | `derived`, which decides what else you fill in (the "population matrix," §2.2): `count` / `sum` need `sql_definition` + `aggregation_method`; `ratio` needs `numerator` + `denominator`; `derived` needs `expr`.
- `grain` + `entity` — which physical table and what's being counted (session / user / order...).
- `dimensions_supported` — the whitelist of dimensions you may slice by; anything else is a hard error.
- `business_definition`, `caveats`, `common_mistakes` — honest plain-English meaning and footguns.
- `guardrails` — machine-readable "how this metric can mislead," which becomes the **Critic's ammunition**.
- `root_cause` — `upstream_drivers`, `downstream_impacts`, `related_dimensions`, `decomposition_path`: this turns the metric into a node in the diagnosis tree.
- `owner`, `version` (SemVer), `testing`, `validation_checks`, `freshness_requirements`.

Authoring vs runtime split (§3): you write *descriptive* names (`metric_name`, `sql_definition`, `aggregation_method`, `dimensions_supported`); the resolver maps them to short internal keys (`name`, `sql`, `agg`, `dimensions`) at load time. Never hand-write the short keys — that's a compile error.

How the six metric-consuming agents use it: Monitor builds daily series and detects anomalies; Decompose splits ΔR along `related_dimensions`; Diagnose walks `upstream_drivers` reconciling and significance-testing each step; Prescribe picks experiment primary/guardrail metrics; Narrator renders the brief using each metric's `label` / `business_definition` verbatim; Critic attacks findings using the `guardrails` block. (The Orchestrator plans but consumes no metrics.)

CI is the firewall (\$6). Four gates a change must clear: (1) **referential-integrity compile** — any dangling `numerator / denominator / expr` /dimension reference fails the build, and the server refuses to start; (2) **reconciliation** $\leq 0.5\%$ (revenue to the cent); (3) **anomaly/data-quality** checks; (4) the **eval gate** — smoke (12 scenarios) every push, full (50 scenarios) on PRs to `main`, requiring $\geq 85\%$ top-1 root-cause accuracy and no > 2 -pt regression.

SemVer governs change (\$4): MAJOR = meaning/number changes (breaking, needs eval re-baseline); MINOR = additive (e.g. adding a dimension or alias); PATCH = cosmetic. Renames go through alias + deprecation — never a silent rename.

Easy things to get wrong

- **Defining a metric anywhere but the YAML.** That breaks invariant I1 immediately.
- **Averaging per-segment rates.** Always `SUM(num)/SUM(den)` after grouping.
- **Using a non-USD money column.** Money uses `*_in_usd` columns only.
- **Silently renaming a metric.** Use alias + deprecation window; renames break the audit trail and eval labels.
- **Violating the population matrix.** A `ratio` with an `expr` set (instead of `numerator / denominator`) fails the compile.
- **Inventing a dimension or an 11th channel group.** Dimensions and the 10 channel groups are a fixed canonical set; synonyms are mapped back before reaching `semantic-mcp` (rule G5).

Glossary — the exact words, demystified

- **semantic layer / registry** — `semantic_layer.yaml`, the one file that defines every metric.
- **governed SQL** — SQL built only from registry templates, never typed by the model.
- **grain** — the physical table/level a metric lives at (`fct_funnel`, `fct_orders` ...).
- **guardrails** — the per-metric "how this can mislead" block the Critic uses to refute.
- **root_cause block** — the per-metric links that make it a node in the diagnosis tree.
- **alias** — a legacy name resolving to the canonical metric (e.g. `transactions` → `orders`); never a second definition.
- **SemVer** — MAJOR.MINOR.PATCH versioning per metric.
- **G1–G5** — grounding rules (no raw SQL; no stats in prose; `dry_run` first; reconcile $\leq 0.5\%$; canonical names only).

When to open the real doc

Open `pdf/docs/architecture/METRIC_GOVERNANCE_GUIDE.pdf` when you need the full field-by-field reference (§2.1), the type→population matrix (§2.2), the resolver field mapping (§3), the SemVer bump rules (§4), the guardrail/RCA block specs (§7–8), or the add/change-a-metric checklist (§10).